JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ChoiceFormat

java.lang.Object
    java.text.Format
        java.text.NumberFormat
            java.text.ChoiceFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

```
public class ChoiceFormat
extends NumberFormat
```

`ChoiceFormat` is a concrete subclass of `NumberFormat` that allows you to attach a format to a range of numbers. It is generally used in a `MessageFormat` for handling plurals. The choice is specified with an ascending list of doubles, where each item specifies a half-open interval up to the next item:

```
X matches j if and only if limit[j] ≤ X < limit[j+1]
```

If there is no match, then either the first or last index is used, depending on whether the number (X) is too low or too high. If the limit array is not in ascending order, the results of formatting will be incorrect. ChoiceFormat also accepts `\u221E` as equivalent to infinity(INF).

Note: `ChoiceFormat` differs from the other `Format` classes in that you create a `ChoiceFormat` object with a constructor (not with a `getInstance` style factory method). The factory methods aren't necessary because `ChoiceFormat` doesn't require any complex setup for a given locale. In fact, `ChoiceFormat` doesn't implement any locale specific behavior.

Patterns

A `ChoiceFormat` pattern has the following syntax:

Pattern:
SubPattern *("|" SubPattern)

SubPattern:
Limit Relation Format

Note: Each additional SubPattern must have an ascending

Limit-Relation interval

Limit:
Number / "∞" / "-∞"

Number:

["-"] *(Digit) 1*(Decimal / Digit) *(Digit) [Exponent]

Decimal:
1*(Digit ".") / 1*("." Digit)

Digit:
0 - 9

Exponent:
*(Digit) Digit ExponentSymbol Digit *(Digit)

ExponentSymbol:
"e" / "E"

Relation:
"#" / "<" / "≤"

Format:
Any characters except the special pattern character '|'

Note:The relation ≤ is not equivalent to <=

To use a reserved special pattern character within a *Format* pattern, it must be single quoted. For example, `new ChoiceFormat("1#'|'foo'|'").format(1)` returns `"|foo|"`. Use two single quotes in a row to produce a literal single quote. For example, `new ChoiceFormat("1# ''one'' ").format(1)` returns `" 'one' "`.

Usage Information

A `ChoiceFormat` can be constructed using either an array of formats and an array of limits or a string pattern. When constructing with format and limit arrays, the length of these arrays must be the same. For example,

- *limits* = {1,2,3,4,5,6,7}
  formats = {"Sun","Mon","Tue","Wed","Thur","Fri","Sat"}
- *limits* = {0, 1, ChoiceFormat.nextDouble(1)}
  formats = {"no files", "one file", "many files"}
  (`nextDouble` can be used to get the next higher double, to make the half-open interval.)

Below is an example of constructing a ChoiceFormat with arrays to format and parse values:

```
double[] limits = {1,2,3,4,5,6,7};
String[] dayOfWeekNames = {"Sun","Mon","Tue","Wed","Thur","Fri","Sat"};
ChoiceFormat form = new ChoiceFormat(limits, dayOfWeekNames);
ParsePosition status = new ParsePosition(0);
```

```
for (double i = 0.0; i <= 8.0; ++i) {
    status.setIndex(0);
    System.out.println(i + " -> " + form.format(i) + " -> "
                               + form.parse(form.format(i),status));
}
```

Below is an example of constructing a ChoiceFormat with a String pattern:

```
ChoiceFormat fmt = new ChoiceFormat(
     "-1#is negative| 0#is zero or fraction | 1#is one |1.0<is 1+ |2#is two |2<is

System.out.println(fmt.format(Double.NEGATIVE_INFINITY)); // outputs "is negative
System.out.println(fmt.format(-1.0)); // outputs "is negative"
System.out.println(fmt.format(0)); // outputs "is zero or fraction"
System.out.println(fmt.format(0.9)); // outputs "is zero or fraction"
System.out.println(fmt.format(1)); // outputs "is one"
System.out.println(fmt.format(1.5)); // outputs "is 1+"
System.out.println(fmt.format(2)); // outputs "is two"
System.out.println(fmt.format(2.1)); // outputs "is more than 2."
System.out.println(fmt.format(Double.NaN)); // outputs "is negative"
System.out.println(fmt.format(Double.POSITIVE_INFINITY)); // outputs "is more tha
```

For more sophisticated patterns, `ChoiceFormat` can be used with `MessageFormat` to produce accurate forms for singular and plural:

```
MessageFormat msgFmt = new MessageFormat("The disk \"{0}\" contains {1}.");
double[] fileLimits = {0,1,2};
String[] filePart = {"no files","one file","{1,number} files"};
ChoiceFormat fileChoices = new ChoiceFormat(fileLimits, filePart);
msgFmt.setFormatByArgumentIndex(1, fileChoices);
Object[] args = {"MyDisk", 1273};
System.out.println(msgFmt.format(args));
```

The output with different values for `fileCount`:

```
The disk "MyDisk" contains no files.
The disk "MyDisk" contains one file.
The disk "MyDisk" contains 1,273 files.
```

See `MessageFormat` for caveats regarding `MessageFormat` patterns within a `ChoiceFormat` pattern.

Synchronization

Choice formats are not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

API Note:

A subclass could perform more consistent pattern validation by throwing an `IllegalArgumentException` for all incorrect cases. See the `Implementation Note` for this implementation's behavior regarding incorrect patterns.

This class inherits instance methods from `NumberFormat` it does not utilize; a subclass could override and throw `UnsupportedOperationException` for such methods.

Implementation Note:

Given an incorrect pattern, this implementation may either throw an exception or succeed and discard the incorrect portion. A `NumberFormatException` is thrown if a `limit` can not be parsed as a numeric value and an `IllegalArgumentException` is thrown if a `SubPattern` is missing, or the intervals are not ascending. Discarding the incorrect portion may result in a ChoiceFormat with empty `limits` and `formats`.

Since:

1.1

See Also:

`DecimalFormat`, `MessageFormat`, Serialized Form

Nested Class Summary

Nested classes/interfaces declared in class NumberFormat
`NumberFormat.Field`, `NumberFormat.Style`

Field Summary

Fields declared in class NumberFormat
`FRACTION_FIELD`, `INTEGER_FIELD`

Constructor Summary

Constructors

Constructor	Description

ChoiceFormat(double[] limits, String[] formats)	Constructs with the limits and the corresponding formats.
ChoiceFormat(String newPattern)	Constructs a ChoiceFormat with limits and corresponding formats based on the pattern.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
void	**applyPattern**(String newPattern)	Apply the given pattern to this ChoiceFormat object.
Object	**clone**()	Overrides Cloneable
boolean	**equals**(Object obj)	Compares the specified object with this ChoiceFormat for equality.
StringBuffer	**format**(double number, StringBuffer toAppendTo, FieldPosition status)	Returns pattern with formatted double.
StringBuffer	**format**(long number, StringBuffer toAppendTo, FieldPosition status)	Specialization of format.
Object[]	**getFormats**()	Returns the formats of this ChoiceFormat.
double[]	**getLimits**()	Returns the limits of this ChoiceFormat.
int	**hashCode**()	Returns the hash code for this ChoiceFormat.
boolean	**isStrict**()	Returns true if this format will parse numbers strictly; false otherwise.
static final double	**nextDouble**(double d)	

		Finds the least double greater than `d`.
`static double`	**`nextDouble`**`(double d, boolean positive)`	Finds the least double greater than `d` (if `positive` is `true`), or the greatest double less than `d` (if `positive` is `false`).
`Number`	**`parse`**`(String text, ParsePosition status)`	Parses the input text starting at the index given by the `ParsePosition` as a `Double`.
`static final double`	**`previousDouble`**`(double d)`	Finds the greatest double less than `d`.
`void`	**`setChoices`**`(double[] limits, String[] formats)`	Set the choices to be used in formatting.
`void`	**`setStrict`**`(boolean strict)`	Change the leniency value for parsing.
`String`	**`toPattern`**`()`	Returns a pattern `string` that represents the `limits` and `formats` of this ChoiceFormat object.
`String`	**`toString`**`()`	Returns a string identifying this `ChoiceFormat`, for debugging.

Methods declared in class NumberFormat

`format`, `format`, `format`, `getAvailableLocales`, `getCompactNumberInstance`, `getCompactNumberInstance`, `getCurrency`, `getCurrencyInstance`, `getCurrencyInstance`, `getInstance`, `getInstance`, `getIntegerInstance`, `getIntegerInstance`, `getMaximumFractionDigits`, `getMaximumIntegerDigits`, `getMinimumFractionDigits`, `getMinimumIntegerDigits`, `getNumberInstance`, `getNumberInstance`, `getPercentInstance`, `getPercentInstance`, `getRoundingMode`,

isGroupingUsed, isParseIntegerOnly, parse, parseObject, setCurrency, setGroupingUsed, setMaximumFractionDigits, setMaximumIntegerDigits, setMinimumFractionDigits, setMinimumIntegerDigits, setParseIntegerOnly, setRoundingMode

Methods declared in class Format

format, formatToCharacterIterator, parseObject

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Constructor Details

ChoiceFormat

```
public ChoiceFormat(String newPattern)
```

Constructs a ChoiceFormat with limits and corresponding formats based on the pattern. The syntax and error related caveats for the ChoiceFormat pattern can be found in the Patterns section. Unlike `ChoiceFormat(double[], String[])`, this constructor will throw an `IllegalArgumentException` if the `limits` are not in ascending order.

Parameters:

`newPattern` - the new pattern string

Throws:

`NullPointerException` - if `newPattern` is `null`

`IllegalArgumentException` - if `newPattern` violates the pattern syntax

See Also:

`applyPattern(String)`

ChoiceFormat

```
public ChoiceFormat(double[] limits,
                    String[] formats)
```

Constructs with the limits and the corresponding formats.

Parameters:

`limits` - limits in ascending order

`formats` - corresponding format strings

Throws:

NullPointerException - if limits or formats is null

IllegalArgumentException - if the length of limits and formats are not equal

See Also:

setChoices(double[], String[])

Method Details

applyPattern

```
public void applyPattern(String newPattern)
```

Apply the given pattern to this ChoiceFormat object. The syntax and error related caveats for the ChoiceFormat pattern can be found in the Patterns section. Unlike setChoices(double[], String[]), this method will throw an IllegalArgumentException if the limits are not in ascending order.

Parameters:

newPattern - a pattern string

Throws:

NullPointerException - if newPattern is null

IllegalArgumentException - if newPattern violates the pattern syntax

See Also:

ChoiceFormat(String)

toPattern

```
public String toPattern()
```

Returns a pattern string that represents the limits and formats of this ChoiceFormat object. The string returned is not guaranteed to be the same input string passed to either applyPattern(String) or ChoiceFormat(String).

Returns:

a pattern string that represents the limits and formats of this ChoiceFormat object

See Also:

applyPattern(String)

setChoices

```
public void setChoices(double[] limits,
                       String[] formats)
```

Set the choices to be used in formatting.

Parameters:

`limits` - contains the top value that you want parsed with that format, and should be in ascending sorted order. When formatting X, the choice will be the i, where limit[i] ≤ X < limit[i+1]. If the limit array is not in ascending order, the results of formatting will be incorrect.

`formats` - are the formats you want to use for each limit.

Throws:

`NullPointerException` - if `limits` or `formats` is `null`

`IllegalArgumentException` - if the length of `limits` and `formats` are not equal

getLimits

```
public double[] getLimits()
```

Returns the limits of this ChoiceFormat.

Returns:

the limits of this ChoiceFormat

getFormats

```
public Object[] getFormats()
```

Returns the formats of this ChoiceFormat.

Returns:

the formats of this ChoiceFormat

format

```
public StringBuffer format(long number,
                           StringBuffer toAppendTo,
                           FieldPosition status)
```

Specialization of format. This method really calls `format(double, StringBuffer, FieldPosition)`. Thus, the range of longs that are supported is only equal to the range that can be stored by double. This will never be a practical limitation.

Specified by:

`format` in class `NumberFormat`

Parameters:

`number` - number to be formatted and substituted.

`toAppendTo` - where text is appended.

`status` - ignore no useful status is returned.

Returns:

the formatted StringBuffer

Throws:

`ArrayIndexOutOfBoundsException` - if either the `limits` or `formats` of this ChoiceFormat are empty

`NullPointerException` - if `toAppendTo` is `null`

See Also:

`Format.format(Object)`

format

```
public StringBuffer format(double number,
                           StringBuffer toAppendTo,
                           FieldPosition status)
```

Returns pattern with formatted double.

Specified by:

`format` in class `NumberFormat`

Parameters:

`number` - number to be formatted and substituted.

`toAppendTo` - where text is appended.

`status` - ignore no useful status is returned.

Returns:

the formatted StringBuffer

Throws:

`ArrayIndexOutOfBoundsException` - if either the `limits` or `formats` of this ChoiceFormat are empty

`NullPointerException` - if `toAppendTo` is `null`

See Also:

`Format.format(Object)`

parse

```
public Number parse(String text,
                    ParsePosition status)
```

Parses the input text starting at the index given by the `ParsePosition` as a `Double`. The value returned is the `limit` corresponding to the `format` that is

the longest substring of the input text. Matching is done in ascending order, when multiple `formats` match the text equivalently in strength, the first matching `limit` is returned. If there is no match, `Double.NaN` is returned.

For example,

```
var fmt = new ChoiceFormat("0#foo|1#bar|2#baz");
fmt.parse("baz", new ParsePosition(0)); // returns 2.0
fmt.parse("quux", new ParsePosition(0)); // returns NaN
```

Specified by:

`parse` in class `NumberFormat`

Parameters:

`text` - the source text.

`status` - an input-output parameter. On input, the status.index field indicates the first character of the source text that should be parsed. On exit, if no error occurred, status.index is set to the first unparsed character in the source text. On exit, if an error did occur, status.index is unchanged and status.errorIndex is set to the first index of the character that caused the parse to fail.

Returns:

A Number which represents the `limit` corresponding to the `format` parsed, or `Double.NaN` if the parse fails.

Throws:

`NullPointerException` - if `status` is `null` or if `text` is `null` and the list of choice strings is not empty.

See Also:

`NumberFormat.isStrict()`

isStrict

```
public boolean isStrict()
```

Description copied from class: NumberFormat

Returns `true` if this format will parse numbers strictly; `false` otherwise.

Overrides:

`isStrict` in class `NumberFormat`

Returns:

`true` if this format will parse numbers strictly; `false` otherwise

Since:

23

See Also:

Leniency Section,
`NumberFormat.setStrict(boolean)`

setStrict

```
public void setStrict(boolean strict)
```

Description copied from class: `NumberFormat`

Change the leniency value for parsing. Parsing can either be strict or lenient, by default it is lenient.

Overrides:

`setStrict` in class `NumberFormat`

Parameters:

`strict` - `true` if parsing should be done strictly; `false` otherwise

Since:

23

See Also:

Leniency Section, `NumberFormat.isStrict()`

nextDouble

```
public static final double nextDouble(double d)
```

Finds the least double greater than `d`. If `NaN`, returns same value.

Used to make half-open intervals.

Implementation Note:

This is equivalent to calling `Math.nextUp(d)`

Parameters:

`d` - the reference value

Returns:

the least double value greater than `d`

See Also:

`previousDouble(double)`

nextDouble

```
public static double nextDouble(double d,
                                boolean positive)
```

Finds the least double greater than `d` (if `positive` is `true`), or the greatest double less than `d` (if `positive` is `false`). If `NaN`, returns same value.

Implementation Note:

This is equivalent to calling `positive ? Math.nextUp(d) : Math.nextDown(d)`

Parameters:

`d` - the reference value

`positive` - `true` if the least double is desired; `false` otherwise

Returns:

the least or greater double value

previousDouble

```
public static final double previousDouble(double d)
```

Finds the greatest double less than `d`. If `NaN`, returns same value.

Implementation Note:

This is equivalent to calling `Math.nextDown(d)`

Parameters:

`d` - the reference value

Returns:

the greatest double value less than `d`

See Also:

`nextDouble(double)`

clone

```
public Object clone()
```

Overrides Cloneable

Overrides:

`clone` in class `NumberFormat`

Returns:

a clone of this instance.

See Also:

`Cloneable`

hashCode

```
public int hashCode()
```

Returns the hash code for this `ChoiceFormat`.

Overrides:

`hashCode` in class `NumberFormat`

Implementation Requirements:

This method calculates the hash code value using the values returned by `getFormats()` and `getLimits()`.

Returns:

the hash code for this `ChoiceFormat`

See Also:

`Object.hashCode()`

toString

```
public String toString()
```

Returns a string identifying this `ChoiceFormat`, for debugging.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this `ChoiceFormat`, for debugging

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this `ChoiceFormat` for equality. Returns true if the object is also a `ChoiceFormat` and the two formats would format any value the same.

Overrides:

`equals` in class `NumberFormat`

Implementation Requirements:

This method performs an equality check with a notion of class identity based on `getClass()`, rather than `instanceof`. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

`obj` - object to be compared for equality

Returns:

`true` if the specified object is equal to this `ChoiceFormat`

See Also:

`Object.equals(Object)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.